TrustBridge matching: method and code

Hand-over note for the teammate taking over the matching algorithm. Branch **augustin/prototype**, file **matching.py** (about 120 lines).

1. What it does

The matching compares one **researcher card** with one **business card** and returns a score out of 100, plus short sentences explaining why. It is deterministic and explainable on purpose: no machine learning, every number can be traced back to a field on the two cards. The first version was written fast and deliberately kept simple, so expect to tune it.

Scores are **never stored**. The seed files in `data/` contain no score; the UI calls `score_pair()` every time a score is displayed, so the number always matches the current cards, including after an edit during the demo. Changing the algorithm therefore changes every screen at once, with no data migration.

2. The six criteria

Each criterion returns a sub-score of **1** (clear match), **0.5** (partial) or **0** (no match), together with a sentence that quotes the real values. Equity & IP is the average of two halves, so it can also be 0.25 or 0.75. The vocabularies and lookup tables live in `model.py`.

Criterion	Function	1	0.5	0
Sector	`_sector`	At least one sector shared	Different sectors but same broad family (`SECTOR_FAMILY`: infrastructure, life, deeptech, digital)	Otherwise
Maturity	`_maturity`	Researcher level inside the business range (e.g. 3 in 2-4)	One level outside	Two levels outside
Equity & IP	`_equity_ip`	Equity: same bracket or either side flexible/negotiable. IP: researcher IP status is in `IP_COMPATIBLE`[business expectation]	Equity: adjacent brackets	Equity: further apart. IP: incompatible. Final = (equity + IP) / 2
Geography	`_geography`	Business accepts Europe-wide or the researcher's country	Outside, but researcher open to remote work or relocation	Otherwise
Engagement	`_engagement`	2+ shared types, or every type the researcher wants	Exactly one shared type	None shared
Resources	`_resources`	Every need covered by what the business brings (`NEED_COVERED_BY`)	Some needs covered	None covered

Note: the researcher card has an `engagement_wanted` field that the original brief did not list. It was added so that the engagement criterion exists on both sides. A full co-founding team need is considered covered when the business offers the `co-found` engagement.

3. From sub-scores to the displayed numbers

On their card, each user sets every criterion to **Must have** (weight 5), **Important** (3) or **Nice to have** (1), stored as text in `criteria_weights` and converted by `weights()`.

```
one_sided = 100 * sum(weight[c] * sub[c]) / sum(weight[c])   # once per side
mutual    = (research_score + business_score) / 2             # shown in the lists
```

Both one-sided scores are shown in the match detail: a 90/40 pair is a very different conversation from a 65/65 one. **Hard filter:** if the maturity gap is above 2 levels, `score_pair()` returns **None** and the UI does not show the pair at all.

Reasons are the three criteria contributing the most, ranked by (researcher weight + business weight) / 2 x sub-score, ignoring criteria that scored 0. **Watch out** is the criterion with the lowest sub-score (ties go to the higher combined weight); it is None when every criterion scored 1.

4. Worked example (seed data)

Elena's self-healing concrete coating (`r1`) against Aurora Deep Tech Fund (`b8`).

Criterion	Sub-score	Elena's weight	Aurora's weight
Sector	1	Must have (5)	Must have (5)
Maturity	1	Important (3)	Important (3)
Equity & IP	1	Important (3)	Must have (5)
Geography	1	Nice to have (1)	Nice to have (1)
Engagement	0	Important (3)	Important (3)
Resources	0.5	Must have (5)	Nice to have (1)
Result		14.5 / 20 = **72**	14.5 / 18 = **81**

Mutual = (72 + 81) / 2 = **76**. Watch out: "No common engagement type" (the fund only invests, Elena wants a pilot, licence or research contract).

Rounding detail: Python's `round()` rounds halves to the even number, so 72.5 becomes 72 and 76.5 becomes 76. Harmless, but worth knowing if you compare with a spreadsheet.

5. Code map

Function	Role
`score_pair(rc, bc)`	Only public entry point. Returns None (filtered) or a dict: mutual, research, business, subs, reasons, watch_out.
`CHECKS`	Dict criterion -> function. Each function takes (rc, bc) and returns (sub_score, sentence).
`weights(card)`	Converts the card's Must have / Important / Nice to have labels into 5 / 3 / 1.
`one_sided(subs, w)`	Weighted average out of 100.
`maturity_gap(rc, bc)`	Distance between the researcher level and the accepted range; also used by the hard filter.

6. How to change it safely

• **Tune a rule:** edit one `_criterion` function. Keep the (score, sentence) return shape; the UI relies on it.

• **Add a criterion:** add it to `model.CRITERIA` and `CRITERIA_LABELS`, write the function, register it in `CHECKS`, and add a default weight to the seed cards. The card forms build their weight selectors from `CRITERIA`.

• **Finer scores:** nothing forces 0 / 0.5 / 1; any value between 0 and 1 works (e.g. share of needs covered).

• **Check the seed matches:** run the snippet below after any change and make sure the demo pairs still look sensible.

```
python3 -c "
import json, matching
r = {c['id']: c for c in json.load(open('data/researcher_cards.json'))}
b = {c['id']: c for c in json.load(open('data/business_cards.json'))}
for m in json.load(open('data/matches.json'))['matches']:
    s = matching.score_pair(r[m['researcher_card_id']], b[m['business_card_id']])
```

```
print(m['id'], s and (s['mutual'], s['research'], s['business']))"
```

7. Known limits and ideas

• Sub-scores are coarse, so several pairs reach 100. Graded scores would spread the ranking.

• Sector families are a hand-made grouping of the 12 sectors; a small similarity matrix would be more precise.

• Equity semantics differ by side (what the researcher would give vs what the business expects to take); worth confirming with the team.

• Free text (`looking_for`, the abstract) is ignored. Text similarity could be added later as a seventh, clearly labelled criterion, keeping the explanation readable.